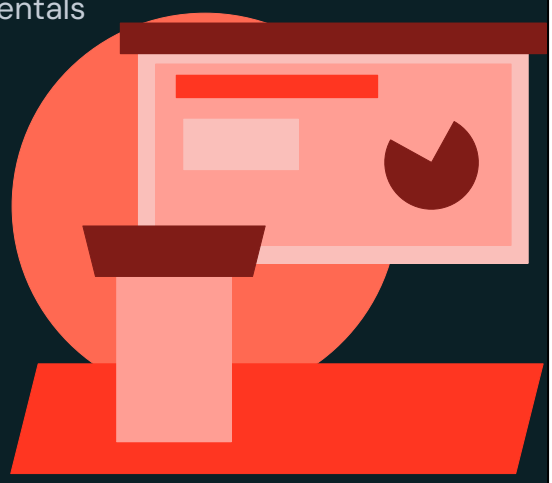
databricks

Lakeflow Spark Declarative Pipeline Fundamentals

LECTURE

Dataset Types Overview



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn how streaming tables, materialized views, and views process data, and how to create each dataset type with SQL—alongside understanding their differences, naming in Databricks, and how pipeline dependencies are automatically managed.

Dataset Types Overview

How streaming tables, materialized views, and views process data

STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**

MATERIALIZED VIEW (MV)

VIEWS

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Within Declarative Pipelines, there are three main dataset types you'll work with:

- Streaming Tables (ST)
- Materialized Views (MV)
- Views

Let's start with Streaming Tables.

Streaming Tables are designed specifically for streaming or incremental data processing. This means they only process new data as it arrives, rather than reprocessing everything each time the pipeline runs. This approach significantly increases efficiency and decreases cost, especially when working with large or frequently updated data.

Dataset Types Overview

STREAMING TABLE Overview

STREAMING TABLE

- Support for **batch** or **streaming** for **incremental** data processing (exactly **once processing** of the files)
- Each time an streaming table is refreshed, data added to the source tables is **appended** to the **streaming table**
- SQL code that creates streaming tables:
  `CREATE OR REFRESH STREAMING TABLE`
- In SQL, you must use `FROM STREAM read_files()` to **invoke Auto Loader functionality** for incremental streaming reads and checkpoints
- File names are **guaranteed to be read only once**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Remember, Streaming Tables support both batch and streaming modes for incremental data processing, ensuring exactly-once processing of your data files.

- Each time a streaming table is refreshed, only the new data added to the source tables is appended, so you always process just the latest data without duplication increasing efficiency.
- In SQL, streaming tables are created or refreshed using the command: CREATE OR REFRESH STREAMING TABLE
- To enable incremental streaming reads with checkpointing, you use the syntax: FROM STREAM read_files()
- This leverages the Databricks Auto Loader functionality, which automatically tracks new files and ensures reliable, incremental ingestion.
- File names are guaranteed to be read only once, with the goal of not having duplicate reads within your incremental ingestion.

Refer to: [Load data with Lakeflow Declarative Pipelines | Databricks on AWS]

Dataset Types Overview

Create a STREAMING TABLE with SQL

Create a STREAMING TABLE named **orders_bronze** from **JSON files**

```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze AS
SELECT
 *,
 current_timestamp() AS processing_time,
 _metadata.file_name AS source_file
FROM STREAM read_files(
   "/Volumes/dbacademy/ops/labuser/orders",
   format => 'JSON');
```

Creates a streaming table named **orders_bronze** in the **1_bronze_db** schema

Use the SELECT clause to specify the columns to read into the streaming table from your data source

- The **STREAM** keyword indicates to use streaming read for the source files (JSON)
- The **read_files()** function reads files under a provided location and returns the data in tabular form

Note: The query is using the default catalog

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's walk through a basic example of how to create a STREAMING TABLE named **orders_bronze** from a set of JSON files in cloud storage.

In this example, we use the CREATE OR REFRESH STREAMING TABLE statement to create a streaming table called **orders_bronze** in the **1_bronze_db** schema.

The SELECT clause defines which columns you want to pull into the streaming table from your source data.

In the FROM clause:

- The STREAM keyword tells the pipeline to use streaming semantics, meaning it will process new files incrementally as they arrive.
- The read_files() function points to the location of your source files. It reads the files and returns the contents in a tabular format.

This setup ensures that only new data is processed each time the pipeline runs, enabling efficient, scalable ingestion directly from cloud storage.

Dataset Types Overview

Create a STREAMING TABLE with SQL

Create a STREAMING TABLE named **orders_silver** that reads from the **STREAMING orders_bronze** table

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver AS
SELECT
  order_id,
  timestamp(order_timestamp) AS order_timestamp,
  customer_id,
  notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

Specify your SQL transformations for the streaming data from the bronze table

Use **STREAM** keyword with the name of the STREAMING TABLE you are reading from to transform new rows

Note: The query is using the default catalog

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now let's take it a step further and create a silver streaming table named **orders_silver**, which reads from the **orders_bronze** streaming table.

The SELECT clause is where you apply your SQL transformation logic to the streaming data. In this example, we're keeping it simple, but in practice these transformations can include complex joins, filters, column derivations, and more.

The most important part is in the FROM clause.Here, you use the STREAM keyword with the name of the source streaming table, in this case, **orders_bronze**. This tells the pipeline to incrementally transform only the new rows that have landed in the bronze table since the last run.

This setup helps ensure that your silver table always stays up to date with the latest processed data, without reprocessing anything that's already been handled.

Dataset Types Overview

How streaming tables, materialized views, and views process data

STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**

MATERIALIZED VIEW (MV)

- Records are processed as required to return accurate results for the **current data state**
- Used for data processing tasks such as:
  - Transformations
  - Aggregations
  - Pre-computing slow queries
  - Frequently used computations

VIEWS

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now let's explore Materialized Views, or MVs, which play a key role in your pipeline.

Materialized Views process records as needed to deliver accurate results based on the current state of your streaming tables. They are designed to automatically keep results up to date as new data flows through upstream tables.

They're ideal for a range of common data processing tasks, such as:

- Applying complex transformations
- Performing aggregations
- Pre-computing slow or expensive queries
- Caching frequently used computations for faster access

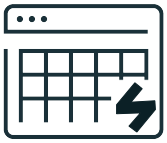
By using MVs, you can improve performance and simplify logic for downstream consumers like dashboards, machine learning models, and analytics tools since the data will be precomputed for your consumers.

Dataset Types Overview

MATERIALIZED VIEW Overview

MATERIALIZED VIEW

- Each time a materialized view is updated, **query results are recalculated to reflect changes in upstream datasets**
- Created and **kept up-to-date** by pipeline
- Use the `CREATE OR REFRESH MATERIALIZED VIEW` syntax
- Can be used **anywhere in your pipeline**, not just in the gold layer
- **Where applicable,** results are **incrementally refreshed for materialized views**, avoiding the need to completely rebuild the materialized view when new data arrives **(Serverless Only)**
- **Incremental refresh for materialized views** is a cost-based optimizer to power fast and efficient transformations for materialized views on Serverless compute



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's take a closer look at how Materialized Views (MVs) work within Declarative Pipelines.

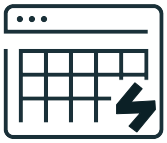
- Each time a materialized view is updated, the query results are recalculated to reflect any changes in the upstream datasets.
- These views are created and maintained automatically by the pipeline, so you don't have to manage manual refresh logic.
- You create them using the standard SQL syntax: CREATE OR REFRESH MATERIALIZED VIEW

Dataset Types Overview

MATERIALIZED VIEW Overview

MATERIALIZED VIEW

- Each time a materialized view is updated, **query results are recalculated to reflect changes in upstream datasets**
- Created and **kept up-to-date** by pipeline
- Use the `CREATE OR REFRESH MATERIALIZED VIEW` syntax
- Can be used **anywhere in your pipeline**, not just in the gold layer
- **Where applicable,** results are **incrementally refreshed for materialized views**, avoiding the need to completely rebuild the materialized view when new data arrives **(Serverless Only)**
- **Incremental refresh for materialized views** is a cost-based optimizer to power fast and efficient transformations for materialized views on Serverless compute



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

- While materialized views are commonly used in the gold layer, they can be used anywhere in your pipeline when performance and reusability matter.
- Where applicable, incremental refresh is used to avoid rebuilding the entire view every time new data arrives. This is currently supported in Serverless compute environments only.
- Behind the scenes, a cost-based optimizer determines whether to incrementally refresh or fully recompute the view. Ensuring fast and efficient performance with minimal resource usage.

Refer to: Incremental refresh for materialized views | Databricks on AWS

Dataset Types Overview

Create a MATERIALIZED VIEW with SQL

Create a MATERIALIZED VIEW named **gold_orders_by_date** that summarizes the **orders_silver** table

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date AS
SELECT
 date(order_timestamp) AS order_date,
 count(*) AS total_daily_orders
FROM 2_silver_db.orders_silver
GROUP BY date(order_timestamp);
```

Create the materialized view **gold_orders_by_date** in the **3_gold_db** schema (database)

Specify the the source table **without the STREAM** keyword to utilize the source STREAMING table
NOTE: Where possible, queries will be incrementally refreshed

Note: The query is using the default catalog

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's take a look at an example of how to create a Materialized View called **gold_orders_by_date**, which summarizes data from the **orders_silver** streaming table.

We use the CREATE OR REFRESH MATERIALIZED VIEW statement to define this view inside the **3_gold_db** schema.

In the FROM clause, we reference the source table, orders_silver, without the STREAM keyword. That's because Materialized Views automatically track changes and manage refreshes based on the upstream streaming table.

Important note: Where possible, the system will use incremental refreshes to update the view efficiently, rather than rebuilding it from scratch. This is supported in Serverless compute and driven by a cost-based optimizer for performance.

This approach is ideal for creating gold-layer aggregations or business-ready outputs with minimal overhead and high performance.

Dataset Types Overview

How streaming tables, materialized views, and views process data

STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**

MATERIALIZED VIEW (MV)

- Records are processed as required to return accurate results for the **current data state**
- Used for data processing tasks such:
  - Transformations
  - Aggregations
  - pre-computing slow queries
  - frequently used computations.

VIEWS

Constructs a virtual table with no physical data based on the query in your Declarative Pipelines

- TEMPORARY VIEW
- VIEW

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Lastly, let's talk about Views in Declarative Pipelines.

Views create virtual tables that do not store any physical data. Instead, they are simply logical representations based on the SQL query defined in your pipeline.

There are two types of views you'll commonly work with:

- TEMPORARY VIEW
- VIEW

Dataset Types Overview

TEMPORARY VIEW Overview

TEMPORARY VIEW

- Temporary views are only **persisted across the lifetime** of the pipeline and **private to the defining pipeline**
- They are **not registered** as an object to Unity Catalog
- Use the `CREATE TEMPORARY VIEW` statement
- TEMPORARY VIEWS are useful as **intermediate queries** that **are not exposed** to end users

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

- Temporary views are short-lived views that exist only for the duration of the pipeline run and are private to the pipeline that defines them.
- These views are not registered as objects in Unity Catalog, so they remain internal and isolated.
- You create them using the SQL statement: CREATE TEMPORARY VIEW
- Temporary views are especially useful for intermediate queries or calculations that support your pipeline logic but don't need to be exposed to end users or other pipelines.

Refer to: CREATE TEMPORARY VIEW (Lakeflow Declarative Pipelines) | Databricks on AWS

Dataset Types Overview

VIEW Overview

VIEW

- Constructs a virtual table with no physical data based on the result-set of a SQL query in your pipelines
- They are **registered** as an object to Unity Catalog
- Use the `CREATE VIEW` statement

LIMITATIONS

- The pipeline must be a Unity Catalog pipeline
- Views cannot have streaming queries, or be used as a streaming source for a pipeline

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

You can also create views within your Declarative Pipelines.

- Views construct a virtual table based on the result set of a SQL query, but they do not store any physical data.
- Unlike temporary views, these views are registered as objects in Unity Catalog, making them available and manageable across your environment.
- To create them, use the SQL statement: CREATE VIEW

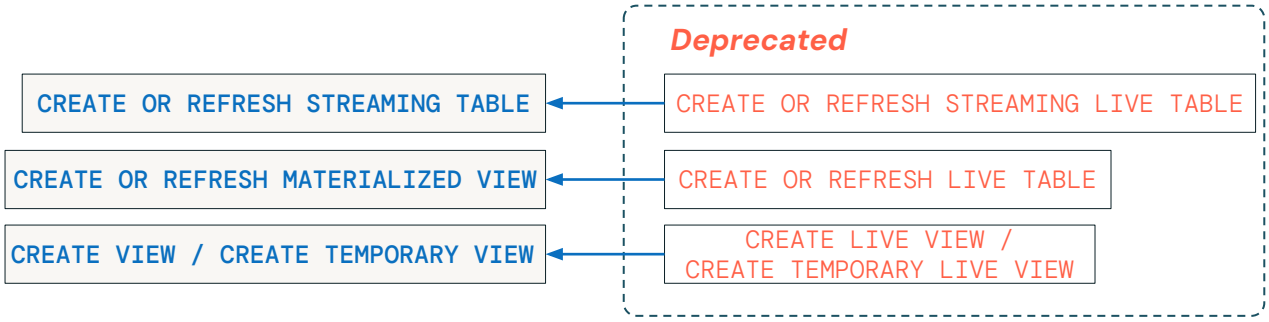
Important Limitations:

- The pipeline must be a Unity Catalog pipeline to create these views.
- Views cannot contain streaming queries and cannot be used as streaming sources within a pipeline.

Refer to: [CREATE VIEW (Lakeflow Declarative Pipelines) | Databricks on AWS]

Dataset Types Overview

Existing users of DLT will notice that we've evolved the names



The **semantics remain the same**, and **we'll support the old syntax** for compatibility. Our goal is to simplify the syntax and match other systems

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

For existing users of DLT, you'll notice that we've updated some names in Lakeflow Spark Declarative Pipelines.

The core semantics and functionality remain the same, so your current workflows will continue to work seamlessly.

We also support the old syntax to maintain backward compatibility during the transition.

Our goal with these changes is to simplify the syntax and better align with industry standards, making it easier to learn and use across different systems.

Dataset Types Overview

Pipeline **dependencies** are parsed automatically, code order is not required

Raw JSON -> Bronze

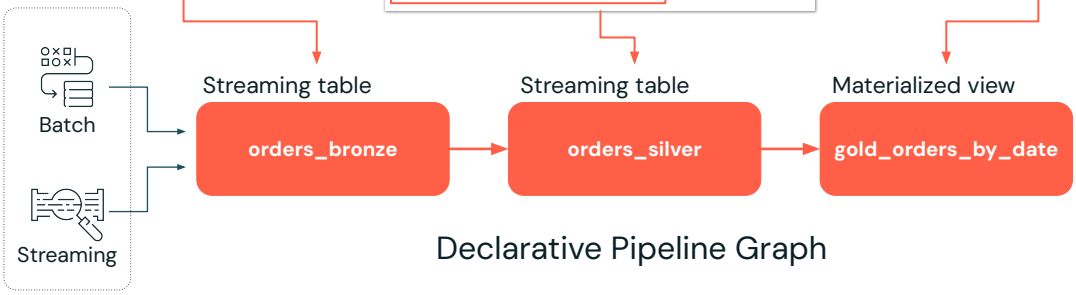
```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze
AS
SELECT
 *,
 current_timestamp() AS processing_time,
 _metadata.file_name AS source_file
FROM STREAM read_files(
   "/Volumes/dbacademy/ops/labuser/orders"
   format => 'JSON');
```

Bronze -> Silver

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
AS
SELECT
  order_id,
  timestamp(order_timestamp) AS order_timestamp,
  customer_id,
  notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

Silver -> Materialized View

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date
AS
SELECT
 date(order_timestamp) AS order_date,
 count(*) AS total_daily_orders
FROM 2_silver_db.orders_silver
GROUP BY date(order_timestamp);
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

One of the powerful features of Declarative Pipelines is that pipeline dependencies are automatically parsed. You don't need to worry about the order of your code.

All dependencies are connected behind the scenes in the Declarative Pipeline Graph, which ensures data flows correctly from one step to the next.

For example:

- The **orders_bronze** table ingests the raw JSON data.
- The **orders_silver** streaming table is linked to orders_bronze, transforming the ingested data incrementally.
- The **gold_orders_by_date** materialized view depends on **orders_silver** streaming table, aggregating the transformed data for downstream business use.

This automatic linking simplifies pipeline development and reduces errors by removing the need to manually manage execution order. It also makes it easy to view and monitor your pipeline and its dependencies, giving you clear visibility into how data flows through each stage.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.